Hyperion: A Highly Effective Page and PC Based Delta Prefetcher

YUJIE CUI, School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, China
WEI CHEN, School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, China
XU CHENG, School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, China
JIANGFANG YI, School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, China

Hardware prefetching plays an important role in modern processors for hiding memory access latency. Delta prefetchers show great potential at the L1D cache level, as they can impose small storage overhead by recording deltas. Furthermore, local delta prefetchers, such as Berti, have been shown to achieve high L1D accuracy. However, there is still room for improving the L1D coverage of existing delta prefetchers. Our goal is to develop a delta prefetcher capable of achieving both high L1D coverage and accuracy. We explore delta prefetchers trained on various types of contextual information, ranging from coarse-grained to fine-grained, and analyze their L1D coverage and accuracy. Our findings indicate that training deltas based on the access histories of both PCs and memory pages for individual PCs and memory pages can lead to increased L1D coverage alongside high accuracy. Therefore, we introduce Hyperion, a highly efficient Page and PC-based delta prefetcher. In terms of the vital component of recording access histories, we implement three different structures and engage in a detailed discussion about them. Furthermore, Hyperion utilizes micro-architecture information (e.g., L1D hits or misses, PQ occupancy) and real-time L1D accuracy to dynamically adjust its issuing mechanism, further enhancing performance and L1D accuracy. Our results show that Hyperion achieves an L1D accuracy of 92.4% and an L1D coverage of 51.9%, along with an L2C coverage of 63.0% and an LLC coverage of 67.5% across a diverse range of applications, including SPEC CPU2006, SPEC CPU2017, GAP, and PARSEC, with a baseline of no prefetching. Regarding performance, Hyperion achieves a 50.1% performance gain, outperforming the state-of-the-art delta prefetcher Berti by 5.0% over baseline across all memory-intensive traces from the four benchmark suites.

CCS Concepts: • **Computer systems organization → Architectures;**

Additional Key Words and Phrases: Hardware prefetch, L1D prefetcher, high accuracy, high coverage

New article, not an extension of a conference paper.
Yujie Cui and Wei Chen contributed equally to this research.
Authors' Contact Information: Yujie Cui, School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, China; e-mail: Yujiecui@pku.edu.cn; Wei Chen, School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, China; e-mail: chenwei_cs@stu.pku.edu.cn; Xu Cheng (Corresponding author), School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, China; e-mail: chengxu@mprc.pku.edu.cn; Jiangfang Yi (Corresponding author), School of Computer Science, Engineering Research Center of Microprocessor & System, Peking University, Beijing, Beijing, China; e-mail: yijiangfang@mprc.pku.edu.cn.

ACM 1544-3973/2024/11-ART68
https://doi.org/10.1145/3675398

ACM Reference Format:
Yujie Cui, Wei Chen, Xu Cheng, and Jiangfang Yi. 2024. Hyperion: A Highly Effective Page and PC Based Delta Prefetcher. *ACM Trans. Arch. Code Optim.* 21, 4, Article 68 (November 2024), 27 pages. https://doi.org/10.1145/3675398

1 Introduction

Due to the speed mismatch between the processor and main memory, the processor can be stalled for hundreds of cycles upon each DRAM access, leading to a degradation in the processor's performance. Modern high-performance processors commonly employ cache hierarchies to bridge the performance gap between the processor and main memory. Data prefetchers play a crucial role in cache hierarchies, effectively hiding cache miss penalties by learning memory access patterns and fetching data into the cache hierarchy in advance. Furthermore, data prefetchers can be deployed across various levels of the cache hierarchy, each with distinct design considerations. In particular, L1D prefetchers are subject to constraints regarding valuable on-chip resources and L1D cache bandwidth, necessitating both minimal storage overhead and high accuracy. On the contrary, prefetchers at the L2 and LLC levels can afford a relatively larger storage overhead and can tolerate some inaccuracy. Nevertheless, the L1D prefetcher can observe unfiltered memory access patterns and a sequence of virtual addresses, which gives it the opportunity to achieve higher performance improvements. Recently, several L1D prefetchers have been proposed, attempting to push the performance limits of prefetching.

The existing L1D data prefetcher, PMP [22], employs a small storage overhead by merging access patterns on memory pages with the same first-accessed page offsets. However, the coarse-grained feature utilized for pattern indexing poses challenges in maintaining high accuracy for benchmark programs that exhibit irregular memory access patterns, consequently leading to a significant consumption of memory bandwidth. The **Instruction Pointer Classifier-based Prefetcher (IPCP)** [27] utilizes three straightforward, lightweight prefetchers to generate prefetch requests based on the IP class. However, IPCP does not account for the timeliness of prefetching. The delta prefetcher is an evolution of the stride prefetcher, and it considers the address difference between the current access and any earlier access, without emphasizing succession [25]. Early delta prefetchers, such as Sandbox and BOP [25, 30], train one best global delta for the entire application, as memory access patterns of different memory pages may be similar. Moreover, MLOP [32] suggests that untimely but accurate prefetch requests can also hide a portion of the miss penalty. Therefore, it trains multiple global deltas with different lookahead steps. However, these prefetchers may issue many useless prefetch requests when the access patterns are different within different memory regions. Berti_DPC3 [31], a prefetcher proposed in DPC3 [15], and Berti [26], one of the state-of-the-art delta prefetchers, train the best local deltas for local program contexts (e.g., memory page, PC). Local delta prefetcher, like Berti, can provide higher L1D accuracy than global delta prefetchers, because the best delta may depend on the local program contexts [26]. For example, the access patterns of two different load instructions might be different.

However, training deltas based solely on one type of fine-grained contextual information[1] may limit the ability to recognize a broader range of memory access patterns. For instance, training local deltas for memory pages might not capture patterns for cases where these pages are accessed by a load instruction with a cross-page constant stride. Therefore, we are motivated to identify

[1]To disambiguate between program context and their respective contextual information, this article designates the access histories of corresponding program context as contextual information. For example, the contextual information of PC refers to access histories associated with individual PCs.

the specific types and optimal quantity of diverse contextual information needed to train deltas, aiming to achieve both high coverage and accuracy. To accomplish this, we initially conducted a series of experiments to train deltas using various types of contextual information, with varying levels of granularity. According to the results, prefetchers that train deltas based on global access histories cannot accurately recognize interleaved patterns. In addition, although prefetchers train deltas based on fine-grained information (e.g., access histories associated with each PC) achieve relatively higher accuracy, there is still room for improvement in their coverage. Therefore, using only one type of contextual information does not provide both high L1D coverage and L1D accuracy. Next, we train deltas based on two types of contextual information that have a low correlation, with at least one providing relatively higher accuracy, and compare the L1D coverage. The results show that the L1D coverage improves most and maintains relatively high accuracy when based on access histories of both individual PCs and memory pages. Furthermore, we take the next step to utilize three types of contextual information but find no notable improvement.

According to our experiments, we propose Hyperion, an L1D delta prefetcher that concurrently trains timely deltas using two types of fine-grained contextual information: access histories of individual PCs and memory pages. Our results demonstrate that Hyperion provides both higher L1D accuracy and greater L1D coverage than existing related prefetchers. To summarize, this article makes the following contributions:

- We are motivated to design a series of experiments to identify the specific types and optimal quantity of diverse contextual information needed to train deltas in a delta prefetcher that achieves both high L1D coverage and accuracy. According to the results, we observe that training deltas concurrently based on access histories of individual PCs and memory pages provides the highest coverage and maintains a relative high accuracy across memory-intensive traces from SPEC CPU2006 and SPEC CPU2017 in our experiments.
- We propose Hyperion, which trains deltas based on access histories of both memory pages and PCs. We have also implemented case design for the Hyperion prefetcher. (1) For the important structure of the Hyperion prefetcher, the history table, we implement and compare three different structures: a structure that consists of two separate tables; a unified history table (DiGHB); and an advanced version of DiGHB with reduced storage. (2) We implement a **unified delta table (U.DT)** to record both deltas of the individual PCs and memory pages by using cache partitioning technique. (3) We implement a confidence computing mechanism to help determine the cache fill level of prefetch requests for improving the accuracy of Hyperion as well as covering more misses.
- We utilize the micro-architecture information (e.g., L1D hits or misses, PQ occupancy) and real-time L1D accuracy to dynamically adjust the issuing mechanism to further improve the performance and L1D accuracy of Hyperion.

Our evaluation shows that Hyperion achieves an average L1D accuracy of 92.4% and an average L1D coverage of 51.9% across 126 memory-intensive traces from SPEC CPU2006, SPEC CPU2017, GAP, and PARSEC with a baseline of no-prefetching. It outperforms the state-of-the-art L1D delta prefetcher, Berti, by 5.0% over baseline for all these memory-intensive traces in a single-core system. In addition, the storage overhead of Hyperion is 4.43 KB, which is less than 10% of the capacity of the L1D cache.

2 Background

Valuable on-chip resources and the constrained bandwidth of the L1D cache require L1D prefetchers to have a small storage overhead and high accuracy. Delta prefetchers show great potential at the L1D cache level given that they can record memory access patterns as deltas with small

storage overhead. Moreover, local delta prefetchers, such as Berti, have been shown to achieve high L1D accuracy. Next, we will introduce recent advances in delta prefetchers and compare the approaches they employ for training deltas.

2.1 Recent Advance in Delta Prefetchers

Delta prefetcher considers the address difference between the current access and any earlier access in access histories, while stride prefetcher focuses on consecutive access differences [26]. The earliest delta prefetcher was proposed by the Sandbox prefetcher [30], which defined an evaluation period to train deltas that could issue accurate prefetch requests. The **Best-offset prefetcher (BOP)** [25] improves upon Sandbox by taking timeliness into account when training deltas. The **Multi-lookahead offset prefetcher (MLOP)**[32] proposes that untimely but accurate prefetch requests could also hide part of the miss penalty and further improves upon the BOP by training global deltas with different lookahead steps. These delta prefetchers primarily train global deltas for the entire application, because the access patterns of different memory pages may be similar.

However, Berti_DPC3 [31], a prefetcher proposed in DPC3, and Berti [26], one of the state-of-the-art delta prefetchers, argue that the best deltas vary with different local program context [26] (e.g., different memory pages and different PCs). Berti_DPC3 evolved from the bit-map prefetchers, which utilized bit vectors to record memory access patterns of spatial memory regions, and considered the prefetch timeliness. However, when the entry for the accessed page is removed from the current table, Berti_DPC3 selects the best delta for this page and records it in the record table. This delta remains unchanged until it is evicted from the record table. This makes Berti_DPC3 fail when there are different or interleaved access patterns in a memory page. However, Berti trains best deltas for each load instruction. Compared to Berti_DPC3, Berti can learn access patterns with cross-page deltas and can also learn many timely deltas for each PC. In addition, by learning deltas for each PC, Berti can apply deltas of a memory page to new memory pages accessed by the same PC. Although the state-of-the-art delta prefetcher, Berti, provides high accuracy, there is still room for improvement in the L1D coverage of Berti.

2.2 The Timeliness-aware Delta Training Mechanisms of Delta Prefetchers

When an access to address $X - d$ triggers a prefetch to address X, the prefetch is deemed timely if the time interval between the accesses to addresses $X - d$ and X exceeds the prefetch latency of X. Additionally, the delta +d is considered timely. For timely deltas, BOP, Berti, and Berti_DPC3 all provide some insights. BOP assumes that the fetch latency of different prefetch addresses is the same, while Berti and Berti_DPC3 consider the fetch latency to be variable due to contention in the **prefetch queue (PQ)** and the **miss status handling register (MSHR)**, as well as the different cache levels at which the miss data resides. Next, we will provide a detailed introduction of how they train timely deltas.

As illustrated in Figure 1(a), BOP first records the base trigger address A of a completed prefetch request for address $A+3$ in the **Recent Request Table (RR Table)**. Second, upon a potential miss at address $A + 4$, BOP retrieves a tested delta, +4, from its pre-defined Delta List. Here, a potential miss refers to a miss that would occur in the absence of prefetching, such as an initial demand hit of a prefetched data or a demand miss. Third, subtracting the tested delta +4 from the access address $A + 4$ yields a match with one of the base trigger addresses A, therefore, the confidence level of delta +4 is incremented. Fourth, the tested delta is updated to the next value in the pre-defined Delta List, namely, +5. Subsequent to issuing a new prefetch, BOP awaits the arrival of the next potential miss to continue its delta training process. In this scenario, when the prefetch request for address $A + 3$ is fulfilled with trigger address A, BOP infers that the prefetch request

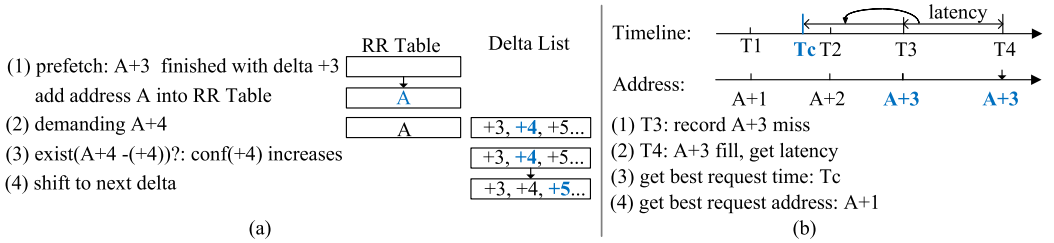

Fig. 1. Timeliness-aware delta training mechanisms of (a) BOP [25], (b) Berti_DPC3 [31], and Berti [26].

for address $A + 4$ can similarly be completed using trigger address A. This assumption implies that the fetch latency for different prefetch addresses originating from the same trigger address remains consistent. However, this assumption may not accurately reflect the actual latency of fetching a miss block. Furthermore, BOP may overlook opportunities to verify timely deltas due to its traversal strategy for the pre-defined Delta List.

Compared to BOP, both Berti_DPC and Berti adopt a different approach by first measuring the fetch latency of potential miss blocks when training timely deltas. This is because the fetch latency can vary due to contention in the PQ and the MSHR, as well as the different cache levels at which the miss data resides. As depicted in Figure 1(b), the complete process consists of four steps for training timely deltas: First, the access address with a timestamp is recorded in the history table: Upon encountering a cache miss, the address of $A + 3$ is recorded along with the timestamp $T3$. If the miss at $A + 3$ is a demand miss, then $T3$ represents the occurrence time of the demand miss; if it is a prefetch miss, then $T3$ represents the time the prefetch request was issued. Second, the fetch latency $T4-T3$ is measured: When the miss data is filled into the cache at time $T4$, the fetch latency is calculated by subtracting the access timestamp $T3$ from the fill timestamp $T4$, and the latency is then stored in the table or cache. Third, the best request time $Tc = T3-(T4-T3)$ is computed: This represents the latest time at which a timely prefetch request can be issued and is determined by subtracting the fetch latency $T4 - T3$ from the demand access timestamp $T3$. Finally, the best request addresses are searched: These addresses, which have access moments earlier than the best request time Tc, are used to calculate timely deltas by subtracting them from potential miss addresses. As shown in Figure 1(b), address $A + 1$ is identified as the best request address, resulting in a timely delta of $+2 = (A + 3) - (A + 1)$. The training process of Berti_DPC and Berti is straightforward, because they measure the fetch latency of potential miss blocks, allowing for accurate calculation of timely deltas by subtracting the best request addresses from the potential miss addresses. Once the timely local deltas are obtained, they are recorded with respective confidence in a table (Delta Table) and can be accessed when the prefetcher is triggered to issue prefetch requests.

3 Motivation

Delta prefetchers can learn deltas based on contextual information with different graininess. However, it is challenging for delta prefetchers that train global deltas to accurately recognize different access patterns. Moreover, training local deltas based on only one type of fine-grained contextual information may miss opportunities to improve coverage. Therefore, we aim to conduct experiments to train delta prefetchers using various types of contextual information, with varying levels of granularity. Through this, we seek to identify the contextual information essential for achieving high L1D coverage and L1D accuracy for a delta prefetcher.

Initially, we evaluate the L1D coverage and accuracy of delta prefetchers, using a single type of contextual information with varying granularity from coarse to fine. Next, we train delta prefetchers on two types of contextual information with low correlation to determine which combination

Table 1. Configuration of the Number of Entries in the History Table and Delta Table for Delta Prefetchers

one type of information	Entries	**two** types of information	Entries	**three** types of information:	Entries
PC	256	PC and global	256+1(length 64)	PC and page and global	256+256+1(length 64)
global	1(length 64)	page and global	256+1(length 64)		
offset	64	PC and offset	256+64		
page	256	page and offset	256+64		
PC+offset	256	PC and page	256+256		
page+offset	256	PC and page+offset	256+256		
PC+page	256	page and PC+offset	256+256		

yields optimal accuracy and coverage. Finally, we extend our evaluation to delta prefetchers utilizing three distinct types of contextual information. For the implementation of these delta prefetchers, we adopt the timeliness-aware delta training mechanism proposed by Berti, as introduced in Section 2.2. Each type of contextual information utilized by the delta prefetchers is paired with a History Table for recording access histories and a Delta Table for recording deltas. Initially, we implement delta prefetchers based on various single types of contextual information. Subsequently, for delta prefetchers utilizing two types of contextual information, we allocate two pairs of History Tables and Delta Tables accordingly. Similarly, delta prefetchers employing three types of contextual information are implemented with three pairs of History Tables and Delta Tables for simplicity. Additionally, all these prefetchers are designed to issue a maximum of 12 prefetch requests per L1D access, prioritizing prefetch requests based on deltas with higher confidence. As depicted in Table 1, each delta prefetcher based on one or multiple types of contextual information is allocated a sufficient and equal number of entries in both the History Tables and the Delta Tables. Each entry in the History Tables typically records 16 access histories, except for prefetchers trained based on the entire application's access history, which records a total of 64 access histories. In the Delta Tables, each entry records 16 deltas. We evaluate these delta prefetchers across all 90 memory-intensive benchmarks from SPEC CPU2006 and SPEC CPU2017, with a baseline of no prefetching. Next, we will provide detailed descriptions of these experiments.

3.1 Experiments on Single Contextual Information

First, we select seven different types of contextual information with different graininess to implement corresponding delta prefetchers. These delta prefetchers, based on different contextual information, are as follows: the global delta prefetcher based on access histories of the entire application, offset based on access histories of pages with identical first-accessed page offsets,[2] PC based on access histories associated with each load instruction, page based on access histories of individual memory pages, PC+offset based on access histories of identical PCs with identical first-accessed page offsets, page+offset based on access histories of identical pages with identical first-accessed page offsets, PC+page associated with identical PCs accessing the identical memory pages. The History Tables and Delta Tables of these prefetchers are indexed by the hash of different program contexts. For example, the History Table and Delta Table of PC+offset are both indexed by the hash of the PC and the first-accessed page offset.

Figure 2 illustrates the L1D coverage and L1D accuracy of delta prefetchers based on various types of contextual information. We will compare and analyze them according to the contextual information from coarse-grained to fine-grained. (1) For delta prefetchers based on coarse-grained information, comparing global with offset, the former exhibits higher L1D coverage and accuracy. This suggests that offset may not discern different patterns as accurately as global. (2) Comparing

[2]According to Reference [22], memory pages with the same first access page offset may have similar memory access patterns.

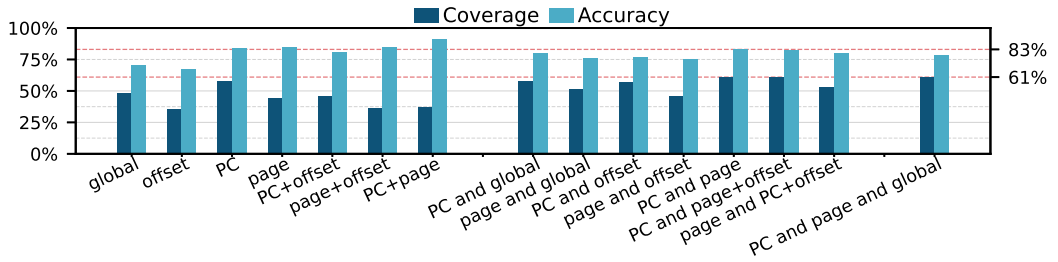

Fig. 2. L1D coverage and L1D accuracy of L1D delta prefetchers based on different types of contextual information over a baseline of no-prefetching across 90 memory-intensive traces from SPEC CPU2006 and SPEC CPU2017. The red lines represent L1D coverage (61%) and L1D accuracy (83%) of the delta prefetcher, PC and page.

delta prefetchers based on coarse-grained information with those based on fine-grained information, the L1D accuracy of global is approximately 17%~18% lower than that of PC and page. This indicates that delta prefetchers based on fine-grained contextual information generally recognize useful patterns more accurately when different patterns are interleaved. Additionally, PC provides 10% higher L1D coverage than global, because it effectively distinguishes between different patterns, leading to more accurate predictions. Comparing delta prefetchers based on different fine-grained contextual information, PC provides 14% higher L1D coverage and slightly lower L1D accuracy than page. This is primarily due to PC's ability to apply the deltas of a memory page to new memory pages accessed by the same PC. Furthermore, PC can learn access patterns involving cross-page deltas. (3) Regarding delta prefetchers PC+offset, page+offset, and PC+page, which are based on more fine-grained contextual information, they do not exhibit higher L1D coverage compared to delta prefetchers PC and page, which rely on fine-grained contextual information. Although PC+offset and PC+page achieve up to 6% higher L1D accuracy than PC, their L1D coverage is 12% to 23% lower. Similarly, compared with page, page+offset provides similar L1D accuracy but lower L1D coverage. This discrepancy arises because, although the first-accessed offsets may differ for a page, the access patterns within this page may be similar or dissimilar. If they are similar, then the confidence of deltas recognized by page+offset will be dispersed across several entries, potentially resulting in missed opportunities to issue more effective prefetch requests. These findings suggest that PC and page effectively identify regular patterns, whereas PC+offset, page+offset, and PC+page, relying on more fine-grained contextual information, may overlook some regular access patterns. Among delta prefetchers that utilize a single type of contextual information, PC achieves the highest L1D coverage at 58%, coupled with a relatively higher L1D accuracy of 84%. However, delta prefetchers based on a single type of contextual information often struggle to adapt to the diverse access patterns across various benchmark programs. For instance, PC outperforms page significantly on benchmark programs featuring fewer load instructions, characterized by regular PCs' patterns, such as 605.mcf_s and 649.fotonik3d_s. Conversely, the performance improvement of page surpasses that of PC notably on benchmark programs with a large number of interleaved load instructions, such as 607.cactuBSSN_s. Therefore, we conduct experiments for prefetchers based on two types of contextual information.

3.2 Experiments on Two or More Types of Contextual Information

Now, we consider employing two different types of contextual information, denoted as I1 and I2, for delta prefetchers. Experimenting on all 21 different combinations of the seven types of contextual information is unnecessary, as our main objective is to achieve both high accuracy and coverage,

which can be roughly estimated beforehand. We propose the following two principles to select two types of contextual information, I1 and I2: (1) at least one of the L1D accuracies of I1 and I2 should be relatively higher; (2) I1 and I2 should have low correlation to generate a high L1D coverage.

Next, we select different combinations according to the preceding two principles. The delta prefetchers Global and Offset, based on coarse-grained information, exhibit relatively lower L1D accuracy. To enhance accuracy, they should collaborate with delta prefetchers that train deltas based on fine-grained or more fine-grained contextual information, such as delta prefetcher PC or page+offset. Moreover, delta prefetchers PC+offset, page+offset, and PC+page based on more fine-grained information provide lower coverage than delta prefetchers PC and page. Therefore, we ultimately pair delta prefetchers Global and Offset with delta prefetcher PC or page for both higher L1D accuracy and coverage. These delta prefetchers based on two types of contextual information are labeled as PC and Global, page and Global, PC and Offset, and page and Offset. To avoid confusion between the delta prefetcher PC + Offset based on a single type of contextual information and PC and Offset based on two types of contextual information, we provide further explanation. The History Table and Delta Table of PC + Offset are indexed by the hash of PC and the first-accessed page offset. In contrast, the two pairs of History Tables and Delta Tables of PC and Offset are indexed by the hash of PC and the hash of the first-accessed page offset, respectively. Regarding delta prefetchers based on fine-grained and more fine-grained information, PC, page, PC+offset, page+offset, and PC+page, they all provide high L1D accuracy. From different types of contextual information they are based on, we should select two types with low correlation to achieve high L1D coverage. Finally, configurations of all delta prefetchers based on two types of contextual information are shown in Table 1.

The L1D accuracy and L1D coverage of delta prefetchers based on two types of contextual information are depicted in Figure 2. Moreover, we have made two observations: (1) Both the L1D coverage and L1D accuracy of the delta prefetchers Global and Offset have shown improvement when they collaborate with PC and page. When Global and Offset cooperate with PC, the L1D coverage of PC and Offset and PC and Global are similar to that of PC, but their L1D accuracy is lower. This suggests that Offset and Global could only recognize a subset of the useful patterns identified by PC, leading to the issue of many useless prefetch requests when different patterns are interleaved. When Global and Offset cooperate with page, the L1D accuracy of page and Offset and page and Global is lower than that of page. However, the coverage of page and Global is higher than that of page due to the presence of regular cross-page patterns, and similar patterns may also exist between different pages. Additionally, the L1D coverage of page and Offset is higher than that of page, because there are also similar patterns across different pages with identical first-accessed page offsets. (2) For delta prefetchers based on two types of fine-grained contextual information, PC and page provides the highest L1D coverage of 61% and L1D accuracy of 83% among all combinations, indicating the presence of different useful patterns in PC and page. The L1D coverage and L1D accuracy of PC and page+offset are similar to those of PC and page. This suggests that although some patterns recognized by page may have been missed by page+offset, they could be identified by PC. Conversely, PC+offset cannot recognize some patterns identified by PC and provides lower L1D coverage than that of PC. Moreover, page cannot recognize these patterns either. Therefore, the coverage of page and PC+offset is relatively lower than that of PC. Overall, PC and page provides the highest coverage of 61% and accuracy of 83% among all combinations based on two types of contextual information, as indicated by the red lines in Figure 2.

Naturally, we should further explore whether combining three types of contextual information leads to higher coverage and accuracy. This entails adding another type of contextual information, alongside access histories of individual PCs and pages, to train the delta prefetcher. The previous

results have indicated that patterns recognized by PC and page+offset and page and PC+offset mostly overlap with patterns recognizable by PC and page. Therefore, we choose to have PC and page cooperate with global, which provides higher L1D accuracy and coverage than offset. However, as Figure 2 demonstrates, there is no improvement in L1D coverage and even a degradation in L1D accuracy compared to PC and page.

In summary, delta prefetchers based on fine-grained or more fine-grained contextual information (e.g., PC and PC+page) exhibit high L1D accuracy, yet there is still room for improvement in their L1D coverage. Hence, we turn our attention to delta prefetchers based on two types of contextual information. Among them, PC and page achieves the most significant improvement in L1D coverage compared to PC, while maintaining relatively high accuracy. Moreover, incorporating three types of contextual information does not yield additional benefits. Based on these findings, we propose Hyperion, which learns timely deltas concurrently based on access histories of PC and page. Next, we will delve into the detailed implementation of Hyperion to further explore its potential.

4 Implementation

To train timely deltas based on access histories of individual PCs and memory pages, we propose Hyperion. There are four important mechanisms with corresponding structures of Hyperion: (1) to train timely deltas, we have adapted Berti's mechanism of measuring data's fetch latency. (2) History Table is an important component of recording access address and timestamp. We implement it in three different structures: The first structure consists of two separate tables, one for recording access histories associated with individual PCs and the other for individual memory pages; the second structure is a unified history table (DiGHB), recording access histories shared by both individual PCs and memory pages; and the third one is an advanced version of DiGHB with less storage than DiGHB. (3) We implement a unified Delta Table to record learned timely deltas. In this table, we transform the indexes of PC and **virtual page number (VPN)** to map entries for PCs and memory pages into different sets. Additionally, we design a confidence evaluation mechanism leveraging counters in the Delta Table to select deltas with high confidence. (4) When it comes to the issue of prefetch requests, Hyperion determines the cache fill level of prefetched data based on the confidence of deltas and the real-time L1D accuracy. Additionally, Hyperion allows prefetch requests based on deltas of the demand PC and accessed memory page to compete for a limited PQ resource. Hyperion issues a maximum of 12 prefetch requests per L1D potential miss, prioritizing prefetch requests based on deltas with higher confidence. These strategies ensure high L1D accuracy of Hyperion. Additionally, when PQ is full, unissued prefetch requests with cache fill levels are stored in a **prefetch buffer (PB)**. These prefetch requests are issued upon a subsequent hit on the potential miss cache line, which is confirmed when the prefetch bit is invalid upon a demand hit at L1D. Next, these mechanisms along with their corresponding structures will be detailed, followed by a comprehensive overview of Hyperion's operational workflow.

4.1 Implementations for Training Timely Deltas

Similar to Berti, we consider that fetch latencies of miss data are variable due to the contention of PQ and MSHR as well as the different cache levels at which miss data reside. Therefore, we measure fetch latency of each potential miss data.

As depicted in Figure 3.1, Hyperion utilizes a History Table to record addresses and timestamps upon an L1D potential miss. Similar to Berti's approach for latency measurement, we extend both the MSHR and PQ with a 16-bit timestamp field. These fields record the timestamp of a demand miss in MSHR or the issue timestamp of a prefetch request in PQ, which serves as the start time of fetched data. In the event of a prefetch request missing in L1D, the issuing timestamp in PQ

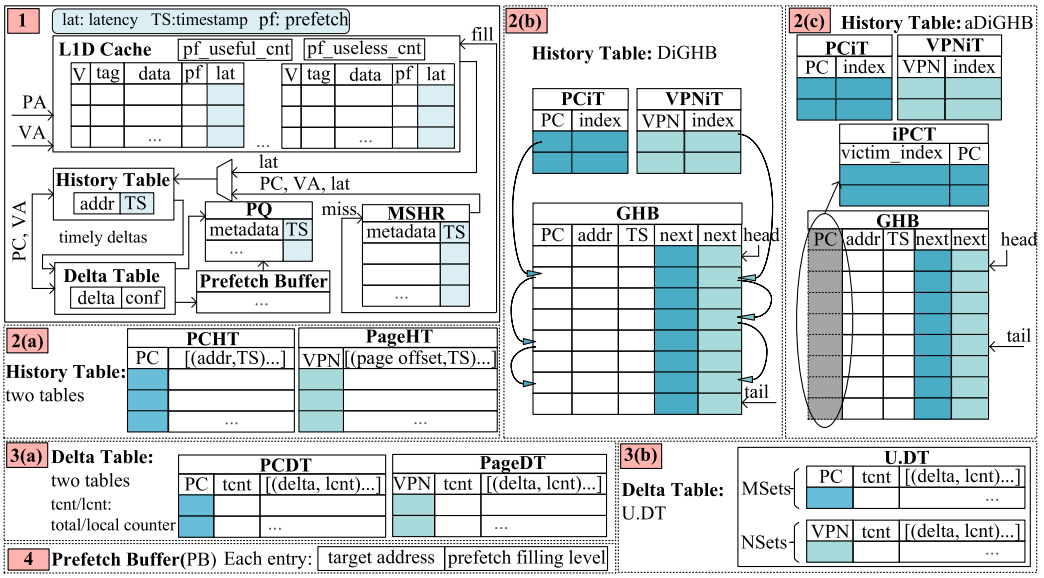

Fig. 3. Details of structures to implement Hyperion. Subfigure 1 presents an overview of Hyperion's hardware structures, with the light blue color indicating components related to the timeliness-aware delta training mechanism. Subfigures 2(a) to 2(c) depict different structures of the History Table, which records access addresses and timestamps for each PC and memory page. Subfigures 3(a) to 3(b) illustrate different structures of the Delta Table, which records timely deltas for each PC and memory page. Subfigure 4 represents the entry for Prefetch buffer (PB), a FIFO buffer.

is transferred to MSHR. Once the missing data are filled into the L1D cache, the filling moment acts as the end time of fetched data. The fetch latency is then computed by subtracting the start timestamp in MSHR from the filling timestamp and stored in the extended 12-bit latency field of the L1D cache line. Upon obtaining the fetch latency of a demand miss, Hyperion promptly searches the History Table, which records the addresses and timestamps of potential misses, to identify the best request addresses for the demanding PC and accessed memory page, as introduced in Section 2.2. Subsequently, Hyperion computes the timely deltas, which are then stored in a Delta Table within entries corresponding to the demanding PC and accessed memory page. However, upon obtaining and storing the latency of a prefetch miss data, the prefetch data's latency is not retrieved from the L1D until the data is first hit. This is because the hit moment represents the actual demand moment of this prefetch data. Following this, similar to a demand miss, Hyperion computes the best request addresses and timely deltas.

4.2 Recording Access Histories in Table for Both PC and Memory Page

History Table is an essential component for recording potential miss information, including demanded addresses and timestamps. However, designing such a table for Hyperion presents challenges, because it needs to be indexed by both PC and VPN. A straightforward structure involves employing two separate tables: the **PC History Table (PCHT)**, indexed by PC, and the **Page History Table (PageHT)**, indexed by VPN, as illustrated in Figure 3.2(a). Each entry of PCHT and PageHT can record up to eight memory access histories. Although this design introduces some storage redundancy, such as recording identical memory access information in both tables, PageHT is indexed by VPN and only needs to record the page offsets of the accessed addresses.

The second structure of the History Table is a unified access history storage structure, as shown in Figure 3.2(b), and we named it the **Double indexed Global History buffer (DiGHB)**. DiGHB consists of three parts: (1) GHB, a cyclic FIFO queue that records the address, timestamp, and associated PC of each potential miss. In addition, each entry also records PC_next and page_next, which are the indexes of the next entries for the associated PC and memory page. Furthermore, there are also two pointers, front and rear, respectively, used to point to the earliest and latest inserted entries in GHB. When $(rear + 1)\%size\_of\_GHB == front$, the GHB is full, and inserting an entry requires the eviction of an existing entry. (2) **PC_index_table (PCiT)**, a PC indexed set-associative cache, records the entry indexes of PCs' earliest accesses in GHB. (3) **page_index_table (VPNiT)**, a VPN indexed set-associative cache, records entry indexes of memory pages' earliest accesses in GHB.

For DiGHB, there are two operations: searching and inserting. The search operation is straightforward. Hyperion first searches the PCiT and VPNiT to locate the earliest entries' position in the GHB for the associated PC and accessed memory page. Then, it extracts the memory access histories corresponding to the PC and memory page, respectively, based on PC_next and page_next in the GHB entries. The search will end when the timestamp of the current access entry is greater than the best requested time, as all subsequent entry timestamps are later than the timestamp of the current access entry. However, the inserting operation is more complex, because the inserting process includes both eviction and insertion. When Hyperion attempts to insert a new entry but the GHB is full, it has to evict the entry that the front pointer of the GHB points to. When evicting an entry from GHB, Hyperion uses its PC and VPN (computed from recorded address) to search the PCiT and VPNiT, and then, respectively, update the corresponding entries using next_PC and next_page within the evicted entry. If the GHB is not full when inserting an entry, then Hyperion does not need to evict an entry. In addition, while inserting a new table entry into GHB, Hyperion will search for the latest table entry inserted under the same PC and the same VPN and update their PC_next and page_next to the index of the newly inserted table entry.

DiGHB utilizes the **first-in-first-out (FIFO)** GHB to record memory access histories. It can obtain memory access histories under different types of program contexts based on the index table and link pointers, such as PC_next and page_next. This makes DiGHB highly scalable, as memory access histories under various types of program contexts can be extracted from the GHB by adding small-capacity index tables and link pointers to the GHB entries. In addition, DiGHB avoids the need to record duplicate memory access information that would be required with two separate tables. However, although DiGHB reduces some redundancy, it also incurs additional storage costs for storing PC_next and page_next. Furthermore, it stores PC information, which is utilized for indexing the PCiT when an entry in the GHB is evicted. However, we have observed that the evicted entry must be the earliest access history in the GHB for a given PC and memory page, due to the FIFO strategy of the GHB. Moreover, the PC information in the GHB entry is only used to index PCiT when the entry is evicted. Therefore, the PC information stored in the GHB is redundant, and we only require an **index_PC_table (iPCT)**, which serves as the index data conversion table for the **PC_index_table (PCiT)**. We propose an advanced DiGHB, named aDiGHB, which is shown in Figure 3.2(c). The aDiGHB employs an iPCT, which is tagged with the entry index of the earliest access and stores the corresponding PC. The iPCT serves as an index-data-conversion table for the PCiT. As a result, Hyperion can eliminate the need to store PC information in the GHB for aDiGHB compared to DiGHB. Moreover, in Section 4.6.1, we will evaluate and compare their performance across memory-intensive traces from SPEC CPU2006, SPEC CPU2017, GAP, and PARSEC when configuring them with the same number of recorded PCs and memory pages, as Table 2 shows.

4.3 Delta Table for Recording Timely Deltas

When it comes to recording deltas in the Delta Table, we also face the challenge of deciding whether to use a single table or two separate tables indexed by PC and VPN. Additionally, as shown in Figure 3.3(a), a simple implementation is also using two delta tables, PCDT and PageDT, to record timely deltas for PCs and memory pages, respectively. To put entries for PCs and memory pages into one table and avoid entries for PC or memory page to be evicted by each other, Hyperion utilizes the technology of cache partitioning. Hyperion implements a unified structure, U.DT, to record deltas for both PC and memory pages, as Figure 3.3(b) shows. The main idea behind U.DT is to enable the PC and VPN indexes to map into distinct and fixed sets of cache entries, which can be implemented by retaining the tag and transforming the index. For example, we allocate M sets to the PC and N sets to the VPN within the Delta table. To enable PC and VPN to index into the ranges $0 \sim (M-1)$ and $M \sim (M+N-1)$, respectively, within the unified Delta table, we can transform key_PC into key_PC' and key_page into key_page', as shown in Equations (1) and (2). Cache partitioning could also be applied to unify the PC_index_table and the page_index_table in DiGHB and aDiGHB. However, we mention it here to provide a detailed explanation.

$$key\_PC' = (key\_PC/M) \times (M+N) + (key\_PC\%M); \tag{1}$$

$$key\_page' = (key\_page/N) \times (M+N) + ((key\_page\%N) + M). \tag{2}$$

As shown in Figure 3.3(b), each entry of U.DT includes tag bits, a total counter, and an array of deltas with corresponding local counters. These counters are used to compute the confidence of each delta, which is defined as the ratio of the delta's local counter to total counter. Using the ratio, rather than the local counter, enables the selection of confident deltas, regardless of whether the PC or memory page is accessed frequently or infrequently. There are two operations for U.DT: updating and searching. For the update operation, Hyperion increments the total counters corresponding to the associated PC and accessed memory page each time it is triggered to train timely deltas. If the total counter reaches its maximum value, then it will be halved as well as all the local counters in this entry. This refreshing strategy can retain high-confidence deltas if the access pattern remains unchanged. After training the timely deltas, the corresponding local counters for these deltas will increase. Moreover, if a new delta is inserted, then the delta with the lowest confidence will be evicted. For the search operation, Hyperion searches the Delta table entries using the PC and VPN, respectively. When the total counter has a small value, deltas with a small local counter value may be considered to have high confidence. Therefore, only if the total counter of the entry is greater than half of its maximum value, Hyperion delivers these high-confidence deltas to compute prefetch target addresses. Otherwise, it will not deliver any deltas. Moreover, each entry in Hyperion's Delta table can record up to 8 pieces of delta information, which is only half of those in Berti's Delta Table entries. When Hyperion uses PC and VPN to search the Delta Table, it can obtain up to 16 pieces of delta information each time.

4.4 Issuing Mechanism

After obtaining timely deltas, Hyperion will sort all the PC's deltas and the memory page's deltas based on their confidence levels. Moreover, Hyperion will preferentially issue prefetch requests for deltas with high confidence, with a limit of up to 12 prefetch requests each time. To avoid polluting the L1D cache as well as missing opportunities to cover misses, Hyperion issues prefetch requests into L1D or L2 cache according to two confidence thresholds, CONF_THRESHOLD_L1D and CONF_THRESHOLD_L2C. We set the default CONF_THRESHOLD_L1D to 0.8 and CONF_THRESHOLD_L2C to 0.2. However, a high CONF_THRESHOLD_L1D of 0.8 may not guarantee high L1D accuracy for some traces. Therefore, Hyperion also refers to the real-time L1D

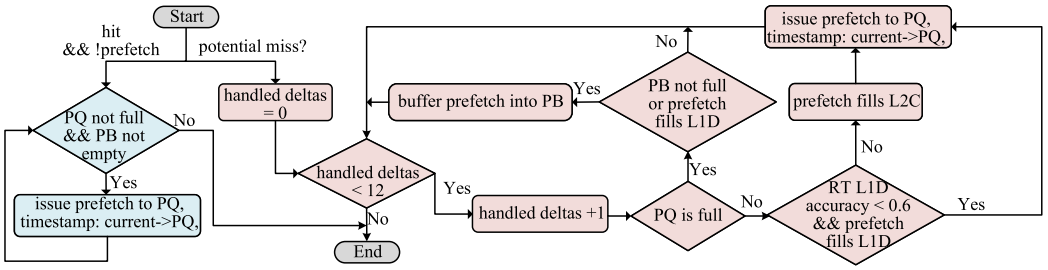

Fig. 4. The issuing mechanism of Hyperion.

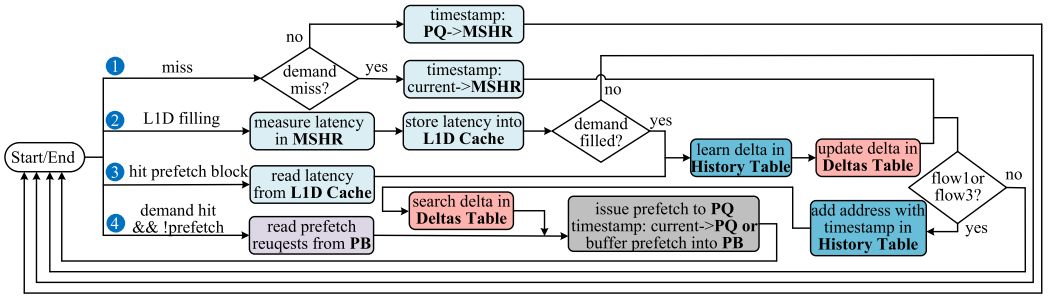

Fig. 5. Four working flows of Hyperion. Operations associated with the timeliness-aware delta training mechanism are represented by light blue blocks, while dark blue blocks denote operations within the History Table. The pink blocks signify operations in the Delta Table, while the gray block pertains to both the issuing mechanism and the timeliness-aware delta training mechanism. Finally, the purple block corresponds to the read operation of the prefetch buffer (PB).

accuracy to determine the cache fill level for prefetch requests. When the real-time L1D accuracy falls below the L1D_ACCURACY_THRESHOLD, Hyperion issues all prefetch requests directly into L2 cache. The real-time L1D accuracy is calculated by dividing the number of useful prefetch requests by the total count of both useful and useless prefetch requests per 10,000 cycles. The number of these requests is recorded by two 64-bit counters: *pf_useful_cnt* and *pf_useless_cnt*. When the first hit occurs for a prefetch block in the L1D cache, the *pf_useful_cnt* is increased by one. When a prefetch block in the L1D is evicted without being used, the *pf_useless_cnt* is increased by one. When it comes to the question of when to prefetch, Hyperion looks up the U.DT for deltas and issues prefetch requests based on these deltas when there is a potential miss. However, if the PQ is full, then some prefetch requests may not have the chance to be issued. Therefore, as shown in Figure 3.4, Hyperion employs a 32-entry PB to record these target addresses along with their corresponding cache fill levels. It issues these buffered prefetch requests upon a subsequent hit on a potential miss cache line, which is confirmed when the prefetch bit is invalid upon a demand hit. The PB employs a FIFO replacement strategy, and Hyperion only inserts prefetch requests with the L1D prefetch level when the PB is full. Generally, the complete issuing mechanism is shown in Figure 4.

4.5 Complete Working Flows of Hyperion

Figure 5 illustrates the complete working process of Hyperion. We have separated the process into four distinct flows, as Hyperion is activated upon the occurrence of any trigger event within these flows and then operates through the following steps. Next, we will introduce each flow in detail:

— Flow 1 starts with an L1D demanding miss or a prefetch miss. In the case of a prefetch miss, the issuing timestamp, initially recorded in the PQ, is subsequently transferred to the MSHR. In the event of a demanding miss, Hyperion records its timestamp in the L1D MSHR. Moreover, Hyperion records the memory access address and the current demand timestamp in the History Table. Additionally, Hyperion searches the Delta Table to select high-confidence deltas for the PC and memory page. Subsequently, based on the confidence levels of these deltas and the real-time L1D accuracy, Hyperion determines the cache fill level for each prefetch request and issues them accordingly. If the PQ is full, then the prefetch target addresses with their prefetch levels are buffered into the PB. When prefetch requests enter the L1D PQ, the issue timestamps of them are also recorded in PQ.
— Flow 2 is initiated when the L1D cache is filled. Hyperion begins by measuring the fetch latency, calculated as the difference between the current moment and the timestamp of a demand miss or prefetch issue recorded in the MSHR. Next, this latency is stored in the corresponding latency field of the L1D cache line. If the data is filled in response to a demand miss, Hyperion will then search the History Table to learn timely deltas and update the Delta table (U.DT).
— Flow 3 begins with the first hit on a prefetched cache line in the L1D cache. Initially, Hyperion retrieves the fetch latency from the latency field of the same cache line. Next, it searches the History Table to learn timely deltas. Then, Hyperion utilizes these timely deltas to update the Delta Table. After that, Hyperion updates the History Table. Finally, Hyperion searches the Delta Table and subsequently issues prefetch requests.
— Flow 4 is triggered when there is a subsequent hit on the potential miss cache line. In this scenario, Hyperion issues prefetch requests recorded in the PB.

4.6 Hardware Overhead

4.6.1 Comparing Different Implementation Structures for Hyperion's History Table. As discussed in Section 4.2, we have explored three distinct structures for implementing Hyperion's History Table. In this section, we will compare their storage overhead and the corresponding speedup of Hyperion across 126 memory-intensive traces from SPEC CPU2006, SPEC CPU2017, GAP, and PARSEC compared to no-prefetching. The configurations, storage requirements, and speedup are detailed in Table 2. DiGHB employs a unified global history buffer to record access histories for both the PCs and the memory pages, achieving a similar speedup to that of using two separate tables. Although DiGHB records each access history once in a FIFO queue, the Two-Table design records addresses just for PCs and only page offsets for memory pages. Moreover, because the number of entries for PCs is fewer than those for memory pages in our configuration, and the Two-Table design does not need to record the next indexes used in DiGHB, the storage requirement of the Two-Table design is smaller than that for DiGHB. However, if the number of entries for PCs significantly exceeds those for memory pages, then DiGHB would require less storage than the Two-Table design. In addition, the Two-Table design provides slightly higher performance than DiGHB. This is because the Two-Table design records once for the same address in the Page History Table, which can accommodate up to 256 different access histories. However, if the same address is accessed by different PCs, then DiGHB has to allocate two entries in its 256-entry **Global History Buffer (GHB)**. Therefore, the number of different access histories recorded in GHB is actually less than that in the Two-Table design in this case. The aDiGHB design compresses the information of PCs within each GHB entry by utilizing an index PC table, which serves as a tag-data reverse table of the PC index table. Moreover, the aDiGHB design reduces DiGHB's storage requirement by 0.28 KB. In summary, all three structures of the History Table offer similar speedups, while the Two-Table design incurs minimal storage overhead. However, there are also

Table 2. The Storage Overhead of Different Structures for the History Table and the Corresponding Speedup of Hyperion over No-prefetching Are Evaluated across 126 Memory-intensive Traces from SPEC CPU2006, SPEC CPU2017, GAP, and PARSEC

Structures	Implementation	Storage	speedup
Two-Table	**Page History table**: 2-set, 16-way cache. Each entry includes an 11-bit tag, a 4-bit LRU, and 8 access histories, each consisting of a 9-bit page offset and a 16-bit timestamp. **PC History table**: 16-way fully associative cache. Each entry includes a 10-bit tag, a 4-bit LRU, and 8 access histories, each consisting of a 24-bit address and a 16-bit timestamp.	1.49KB	50.10%
DiGHB	**page index table**: 2-set, 16-way cache. Each entry includes an 11-bit tag, a 4-bit LRU, and an 8-bit index of GHB. **PC index table**: 16-way fully associative cache. Each entry includes a 10-bit tag, a 4-bit LRU, and an 8-bit index of GHB. **GHB**: 256-entry queue with an 8-bit head pointer and an 8-bit tail pointer. FIFO replacement strategy. Each entry includes a 10-bit PC, a 24-bit address, a 16-bit timestamp, an 8-bit next index for the PC, and an 8-bit next index for the memory page.	2.20KB	49.74%
aDiGHB	**page index table**: 2-set, 16-way cache. Each entry includes an 11-bit tag, a 4-bit LRU, and an 8-bit index of GHB. **PC index table**: 16-way fully associative cache. Each entry includes a 10-bit tag, a 4-bit LRU, and an 8-bit index of GHB. **Index PC table**: 16-way fully associative cache. Each entry includes an 8-bit index of GHB as tag and a 10-bit PC tag as data. **GHB**: 256-entry queue. FIFO replacement strategy. Each entry included a 24-bit address, a 16-bit timestamp, an 8-bit next index for the PC, and an 8-bit next index for the memory page.	1.92KB	49.74%

other disadvantages of DiGHB and aDiGHB, such as longer latency in looking up compared to the Two-Table design. When there are successive access requests for the GHB, we could buffer these requests, but this approach may lead to performance degradation. Therefore, the tradeoffs between performance and storage requirements vary under different circumstances. We have compared these three structures to aid hardware designers in selecting the most suitable implementation. Ultimately, we have chosen the Two-Table design to implement the History Table structure for Hyperion.

4.6.2 Total Storage Overhead of Hyperion. Table 3 lists the default configuration of Hyperion and the storage overhead for each structure. Because Hyperion leverages the similarity of access patterns between adjacent memory pages, the logical page size is set to 32 KB, which is eight times the size of a 4 KB memory page. The storage requirement for Hyperion is just 4.43 KB.

5 Evaluation

5.1 Methodology

5.1.1 Platform and Configurations. We evaluate the prefetchers on a trace-driven simulator ChampSim [11], used for the Second and Third Data Prefetching Championships (DPC2 [14] and

Table 3. Detailed Storage Overhead of Hyperion

Structures	Implementation	Storage
History Table	**Two Tables** refer to implementation of Two Tables in Table 2.	1.49KB
Delta Table	**U.DT**: 5-set, 16-way cache. Set 0 is allocated for PCs and sets 1~4 are allocated for pages. Each entry includes a 10-bit tag, a 4-bit total counter, a 4-bit LRU, and an array of 8 deltas (a 13-bit delta and a 4-bit local counter).	1.50KB
PB	A FIFO buffer with 32 entries. Each entry includes a 57-bit prefetch target address and 1-bit prefetch level.	0.23KB
Extended field	PQ: 16-bit timestamp for each entry with total 16 entries; MSHR: 16-bit timestamp for each entry with total 16 entries; L1D Cache: 12-bit latency for each cache line with total 768 cache lines.	1.19KB
Counters in L1D	A 64-bit useful_counter and a 64-bit useless_counter for L1D prefetch requests.	0.02KB
Total		4.43KB

Table 4. Configuration of Simulated System

Cores	1 to 4 out-of-order cores, 4 GHz, 6-issue width, 4-retire width, 352-entry ROB, 128-entry LQ, 72-entry SQ, 4 KB page
TLBs	64-entry L1ITLB/L1DTLB, 2048-entry STLB
L1I	32 KB, 8-way, 2 cycles, 16-entry PQ, 8-entry MSHR
L1D	48 KB, 12-way, 3 cycles, 16-entry PQ, 16-entry MSHR
L2C	512 KB, 8-way associative, 10 cycles, non-inclusive
LLC	2 MB/core, 16-way, 20 cycles, non-inclusive
DRAM controller	One channel/4-cores. 6400 MTPS, reads prioritized overwrites
DRAM	4 KB row-buffer per bank, open page, burst length 16

DPC3 [15]) and recently modified to support address translation to evaluate prefetchers. The details of the configurations are shown in Table 4.

5.1.2 Evaluated Prefetchers. We compare Hyperion with the winning prefetcher of the DPC3 [15], IPCP [27], the state-of-the-art global delta prefetcher, MLOP [32], and the state-of-the-art local delta prefetcher, Berti [26]. IPCP [27] is a composite prefetcher that classifies PCs into three types: constant strides, complex strides, and global streaming. For each type, IPCP uses a lightweight prefetcher to handle the respective access patterns. MLOP [32], evolved from BOP [25], argues that using one best global delta may miss opportunities to cover cache misses, and that untimely prefetches can also hide part of the long memory access latency. Therefore, MLOP utilizes multiple lookahead tables to learn the best global deltas for different lookahead steps, achieving a higher performance gain than BOP. Berti, a state-of-the-art local delta prefetcher, argues that the best deltas depend on the local program context and trains timely deltas for each PC. Since these prefetchers have been implemented in ChampSim, it is convenient for us to perform fine-tuning on them, and the configurations are shown in Table 5. The detailed configuration of Hyperion was determined after a design space exploration and described in Table 3.

5.1.3 Workloads. We evaluate single-core performance for benchmarks from SPEC CPU2006 [38], SPEC CPU2017 [39], GAP [5], and PARSEC [9]. The traces we use for SPEC CPU2006 and SPEC CPU2017 are provided by DPC-2 [14] and DPC-3 [15], respectively. GAP is a graph processing benchmark suite, and PARSEC is designed for performance studies of multiprocessor machines.

Table 5. Configurations of Evaluated Prefetchers

MLOP	384-entry AMT, 500-update, 16-degree
IPCP	128-entry IP table, 8-entry RST table, and 128-entry CSPT table
Berti	128-entry, 8-way history table, 16-entry, 16-deltas/entry delta table

Table 6. The Number of Traces with LLC MPKI ≥ 1.0 for Each Benchmark Suite

Suite	SPEC CPU2006	SPEC CPU2017	GAP	PARSEC	Total
Number of traces with MPKI>1.0	45	45	20	16	126

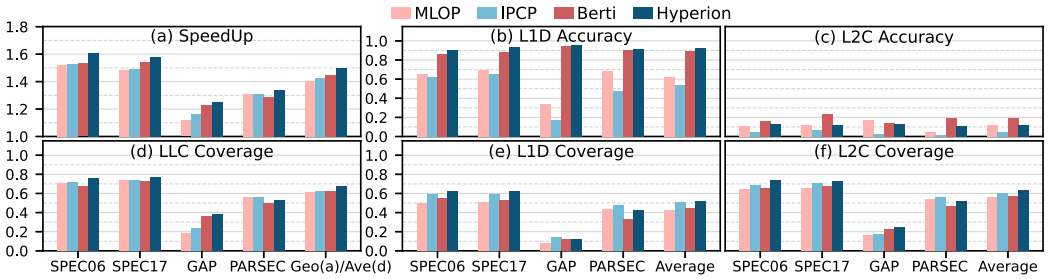

Fig. 6. L1D prefetchers in a single-core system. The shared X label, "Geo(a)/Ave(d)," in subfigure (d) represents the geometric mean speedup for subfigure (a) and the average LLC coverage for subfigure (d).

Moreover, the traces we use for GAP and PARSEC are provided by Berti and Pythia, respectively. For traces with the MPKI no greater than 1.0, even an ideal prefetcher with 100% coverage may not yield a notable performance improvement. Therefore, we evaluate the performance of prefetchers using memory-intensive traces that exhibit at least one LLC miss per kilo-instruction (MPKI ≥ 1.0). The number of traces used for each benchmark suite is shown in Table 6. In multi-core systems, we allocate one trace to each core. For homogeneous trace evaluations, each core uses the same trace, and we employ all the memory-intensive traces used in the single-core evaluation. For heterogeneous trace evaluations, we randomly mixed these traces used in single-core systems to generate 120 heterogeneous traces, each consisting of four single-core traces.

5.1.4 Experimental Scheme. First, we evaluate L1D prefetchers in single-core systems in Section 5.2. Second, we provide insight into the differences between Hyperion and Berti in Section 5.3. Third, we evaluate Hyperion with different design choices in the single-core system in Section 5.4. Fourth, we evaluate the memory traffic of different L1D prefetchers in the single-core system in Section 5.5. Finally, we evaluate the performance of L1D prefetchers in a 4-core system in Section 5.6.

In both single-core and multi-core systems, we use the first 20M trace instructions to warm up caches and the next 80M trace instructions to evaluate the performance. Moreover, in multi-core systems, each core will replay its own evaluation instructions until all the cores have finished the simulation.

5.2 Performance on the Single-core System

5.2.1 Performance of L1D Prefetchers. Figure 6(a) shows the performance speedup of the four state-of-the-art L1D prefetchers over no-prefetching in a single-core system. Among the evaluated prefetchers, Hyperion provides a geometric mean performance gain of 50.1% over no-prefetching across all four benchmark suites, which indicates that Hyperion is efficient in a wide range of

applications. For SPEC CPU2006 and SPEC CPU2017, Hyperion provides performance gains of 60.9% and 58.1%, respectively. In contrast, the performance gain decreases to 25.3% for GAP and 34.0% for PARSEC, as there are more regular access patterns in SPEC CPU2006 and SPEC CPU2017 than in GAP and PARSEC. Hyperion outperforms Berti by 7.7% in SPEC CPU2006; however, the advantage is constrained to 3.6% for SPEC CPU2017. Moreover, the system has an average LLC MPKI of 6.8 across SPEC CPU2006 and an average LLC MPKI of 3.1 across SPEC CPU2017 when utilizing the Berti prefetcher. Consequently, there is greater potential for Hyperion to achieve improvement in SPEC CPU2006 than in SPEC CPU2017. PARSEC is a benchmark suite designed for evaluating the performance of multi-core systems, with its traces generated from executing multi-threaded applications. Therefore, the timeliness of the prefetcher is less critical for PARSEC than for SPEC CPU2006, SPEC CPU2017, and GAP. In addition, different threads generate many interleaved load instructions. Therefore, entries for different PCs are frequently evicted and inserted, resulting in lower performance for Berti in PARSEC. However, MLOP detects access patterns in the global address stream and achieves higher performance improvements than Berti. Moreover, Hyperion can accurately recognize access patterns for memory pages as well as PCs, and it provides the highest performance improvement. As for GAP, we have observed that many benchmark programs exhibit quite chaotic memory access patterns. However, Berti tracks the access pattern of each PC separately and does not directly apply the deltas learned from one PC to others, thereby achieving high prefetch accuracy. However, MLOP struggles to learn the global delta from irregular access patterns, consequently issuing fewer prefetch requests. Moreover, the GS component of IPCP issues an excessive number of unnecessary prefetches, resulting in lower accuracy and diminished performance gains. Therefore, while Berti's performance on the PARSEC benchmark is not as good as that of MLOP and IPCP, it achieves a greater performance improvement on the GAP benchmark. However, Hyperion trains timely deltas using multiple types of contextual information and can adapt to the diverse access patterns of numerous benchmark programs. Consequently, it achieves higher performance improvements compared to the other prefetchers evaluated in both GAP and PARSEC. Overall, Hyperion outperforms MLOP, IPCP, and Berti by 9.4%, 7.8%, and 5.0%, respectively, over no-prefetching, across a wide range of workloads, mainly due to its utilization of multiple types of finer-grained contextual information.

5.2.2 Coverage and Accuracy. Because Hyperion issues prefetch requests to both the L1D cache and the L2 cache, we evaluate the coverage and accuracy for both levels. Moreover, since some prefetch requests may be evicted into LLC, we also evaluate the LLC coverage. As shown in Figures 6(b) through 6(f), Hyperion achieves the highest average L1D coverage of 51.9%, L2C coverage of 63.0%, and LLC coverage of 67.5% across all benchmark suites. This is mainly attributed to its utilization of multiple types of contextual information. Hyperion also achieves the highest average L1D accuracy of 92.4%, which is 30.0%, 38.2%, and 3.4% higher than that of MLOP, IPCP, and Berti, respectively, across all benchmark suites. This is because: (1) Hyperion utilizes fine-grained contextual information to train local deltas. (2) It only uses high-confidence deltas to generate prefetch requests. (3) It filters out inaccurate prefetch requests directed to the larger L2 cache based on real-time L1D accuracy. In contrast, MLOP is unable to accurately recognize interleaved patterns. For example, in the 605.mcf_s-782 trace from SPEC CPU2017, three PCs (0x4049de, 0x4049e5, and 0x4049cc) are responsible for the majority of L1D accesses [26]. The interleaving of access by these PCs poses significant challenges to the training of global deltas within MLOP. Moreover, MLOP generates prefetch requests based on the best delta with each lookahead step without taking into account the associated confidence level of that delta. For regular access patterns, the CS component of IPCP can achieve high prefetch accuracy. However, for complex access patterns, achieving high accuracy is challenging for the CPLX component of

Table 7. The Traces with Lowest or Highest Performance Gain of Hyperion over No-prefetching

Suite	trace with lowest speedup	lowest gain	trace with highest speedup	highest gain
SPEC CPU2006	429.mcf-192B	−0.19%	429.mcf-217B	242.84%
SPEC CPU2017	605.mcf_s-1536B	−3.89%	602.gcc_s-2226B	360.63%
GAP	pr-14	0.18%	bfs-10	79.59%
PARSEC	parsec_2.1.streamcluster.simlarge. prebuilt.drop_0M.length_250M	−1.03%	parsec_2.1.streamcluster.simlarge. prebuilt.drop_250M.length_250M	117.61%

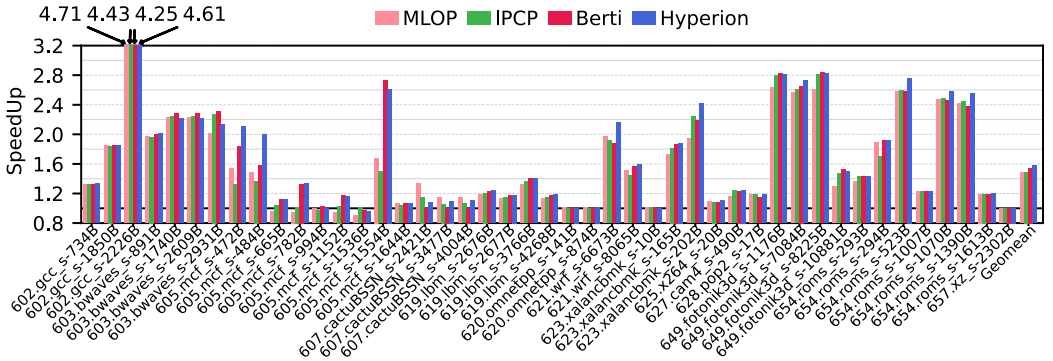

Fig. 7. Speedup of evaluated L1D prefetchers for individual trace across SPEC CPU2017 over no-prefetching.

IPCP, and the GS component tends to issue a large number of useless prefetches. Although Berti also achieves high prefetch accuracy, Hyperion utilizes the real-time L1D accuracy to filter out inaccurate prefetch requests, thereby achieving higher accuracy.

For SPEC CPU2006 and SPEC CPU2017, Hyperion achieves the highest coverage among the evaluated prefetchers by leveraging multiple types of contextual information and effectively identifying diverse access patterns across various benchmark programs. In addition, the local delta prefetcher Berti also achieves relatively high coverage. However, as shown in Figure 7, in a benchmark program such as CactuBSSN, hundreds of load instructions are interleaved and executed. This complexity poses a challenge for Berti in tracking the local behavior of each load instruction within the benchmark. However, Hyperion trains timeliness deltas separately based on multiple types of contextual information, achieving higher performance improvements than Berti in the CactuBSSN benchmark. When faced with irregular access patterns in GAP and PARSEC, Hyperion still provides higher coverage than Berti, which indicates training local deltas based on access histories of memory pages, in addition to those of PCs, is effective across a wide range of applications. However, Hyperion does not cover as many misses as IPCP and MLOP for the PARSEC benchmark suite. This is because Hyperion only utilizes high-confidence deltas to generate prefetch requests, thereby missing out on some prefetch opportunities. However, the GS prefetcher of IPCP issues a large number of prefetch requests, which leads to high coverage but at the cost of accuracy. Moreover, MLOP uses global deltas for the entire application without distinguishing between different PCs and memory pages, allowing it to achieve higher coverage as well. As shown in Figure 6, Hyperion achieves significantly higher accuracy than MLOP and IPCP.

5.2.3 Detailed Speedup of Prefetchers for Individual Traces. We present the traces that provide the highest or lowest speedup over no-prefetching for each benchmark suite, as shown in Table 7. Due to space limitations, we present the speedup of individual traces only for SPEC CPU2017, as shown in Figure 7. Among the evaluated prefetchers, Hyperion achieves the highest performance gain across SPEC CPU2017.

For the trace 605.mcf_s-484B, Hyperion surpasses Berti by 41.7%, IPCP by 62.4%, and MLOP by 50.5% with a baseline of no-prefetching. When analyzing the accesses in the trace 605.mcf_s-484B, we find that most PCs exhibit irregular access patterns. For instance, the load instruction with PC 0x40307e is one that accesses the L1D cache with a high number of potential misses. The sequence of its stride values includes 7,771,691, 15,543,385, −30,965,339, 849,996, 971,424, 1,942,848, 3,885,698, 7,771,396, and so on. Therefore, Berti and IPCP fail to identify regular patterns for PC 0x40307e. However, the virtual page 0x7efef180e, accessed by PC 0x40307e, exhibits a repeated stride sequence −1, 1, 1, −1, 1, −1, and so on. Although IPCP's GS prefetcher records the footprint of frequently accessed pages, it does not consider the timeliness of prefetching. Moreover, MLOP utilizes a global delta for prefetching, which is affected by the interleaving of various access patterns. For example, virtual page 0x7efef180 exhibits a repeated stride sequence $-1, 1, 1, -1, 1, -1, \ldots$, and virtual page 0x7efef1634 shows a different repeated stride sequence $-29, 27, -28, -1, -31, -29, 27, -28, -1, -31, \ldots$. In this case, because MLOP does not distinguish between the access patterns of different memory pages, it issues many useless prefetch requests. For the trace 603.bwaves_s-2931B, all evaluated L1D prefetchers provide a notable performance improvement over no prefetching, ranging from 101.4% to 131.1%. Although many PCs and memory pages in the trace 603.bwaves_s-2931B exhibit regular access patterns, some memory pages are accessed in an interleaved manner by a few PCs. In addition, these PCs exhibit a frequent stride of +64. Therefore, it is challenging for MLOP to recognize this pattern, resulting in a 14.0% lower speedup compared to Berti over no-prefetching. Hyperion also provides 15.7% less speedup than Berti over no-prefetching for the trace 603.bwaves_s-2931B. This is because the history table of Hyperion records access histories from at most 16 different PCs, which is fewer than the number Berti can record. Moreover, the access patterns in the trace 605.mcf_s-1536B are irregular, resulting in performance degradations of 9.8%, 2.9%, and 3.9% for MLOP, Berti, and Hyperion, respectively, compared to no-prefetching. However, IPCP provides a 1.44% performance gain for the trace 605.mcf_s-1536B due to its GS prefetcher.

5.3 An Insight into Hyperion's Difference with Berti

We have explained the motivation for designing a delta prefetcher to achieve both high coverage and accuracy. According to the introduction in Section 3, we propose Hyperion, which trains timely deltas based on two types of finer-grained contextual information: the access histories of both the PC and the memory page. Berti is one of the state-of-the-art local delta prefetchers that trains timely deltas for each PC. In this section, we aim to provide an insight into the differences between Hyperion and Berti. We analyzed the traces based on the evaluation results and identified two primary situations in which Hyperion outperforms Berti. Situation (1): When certain PCs exhibit irregular access patterns, some memory pages targeted by these accesses may actually maintain relatively regular access patterns. An example is trace 605.mcf_s-484B, which is introduced in Section 5.2.3. This might be the case of pointer-chasing, where the patterns of individual PCs are irregular when accessing different pointer nodes. However, the data structure at each pointer node is fixed, and the access histories of each pointer node exhibit regular patterns. Therefore, more regular memory access behavior may be observed by training deltas based on the access history of the same memory page. Situation (2): We find that many demand accesses issued by a large number of different PCs often fall into a few pages. For example, in the trace 607.cactuBSSN_s-2421B, there are 2,000 successive potential misses demanded by 1,778 different PCs, but distributed across only 172 different memory pages. In this trace, Berti provides a 0.1% performance gain, while Hyperion provides a 7.7% performance gain over no-prefetching. This issue appears to be resolved by increasing the table size of Berti. Therefore, we designed experiments to increase the size of Berti's history table and delta table by four times, which requires increasing the storage to 6.5 KB.

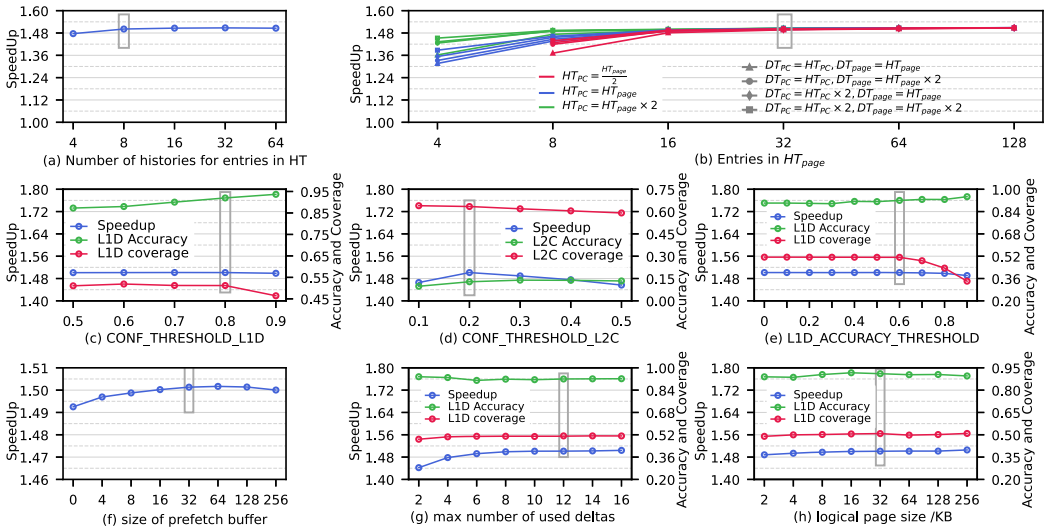

Fig. 8. Performance, L1D accuracy, L1D coverage, L2C accuracy, and L2C coverage of different design choice of Hyperion.

However, compared to the original Berti, the performance gain of the enhanced Berti for trace 607.cactuBSSN_s-2421B is negligible. However, Hyperion trains timely deltas based on multiple types of contextual information, resulting in enhanced adaptability and significant performance improvements.

5.4 Analysis of Different Design Choice of Hyperion

To explore the the benefit of different design choice of Hyperion. We do a series of experiments for all the memory-intensive traces across the four benchmark suites. To accelerate the evaluation, we use the first 50M trace instructions to warm up the caches and the next 20M trace instructions to evaluate the performance.

5.4.1 The Maximum Number of Access Histories Recorded by Each History Table Entry. As shown in Table 2, each entry in Hyperion's history tables (PCHT and PageHT) can record up to 8 memory access histories. We evaluated the impact of the number of access histories per table entry on the performance of Hyperion. As shown in Figure 8(a), when the number of histories increases from 4 to 8, the performance gain of Hyperion over no-prefetching ranges from 47.6% to 50.1%. However, when the number of histories is greater than 8, there is no significant performance improvement. Therefore, we have selected 8 access histories per table entry as the default configuration for Hyperion.

5.4.2 Different Number of Recording PCs and Memory Pages. To determine the appropriate number of entries for PCs and memory pages in the History Tables and Delta Table, we conducted a series of experiments. The Delta Table primarily records trained timely deltas, and Hyperion issues prefetch requests based on this table. The number of entries for PCs and memory pages in the delta table, denoted as DT_{pc} and DT_{page}, is set to be equal to or two times that in the history table, denoted as HT_{pc} and HT_{page}. In addition, we adjust HT_{pc} to be half, equal to, or twice the value of HT_{page} to reduce the design space. In total, there are 12 different combinations of HT_{pc}, DT_{pc}, and DT_{page} for each HT_{page}. When HT_{page} is set to 4, we do not evaluate the performance

when HT_{pc} is half of HT_{page} because, in this situation, there are only two entries for PCHT, which is inadequate.

The performance shows an upward trend as HT_{page} increases from 4 to 16 in the horizontal orientation, as depicted in Figure 8(b). Likewise, in the vertical orientation, when HT_{page} is between 4 and 16, increasing HT_{pc}, DT_{pc}, and DT_{page} also leads to performance improvements. Moreover, performance remains relatively stable when the number of entries for memory pages exceeds 16 and the number of entries for PCs exceeds 8 in the history and delta tables. To strike a balance between storage overhead and performance, we have ultimately chosen the following values: $HT_{page} = 32$, $HT_{pc} = 16$, $DT_{page} = 64$, and $DT_{pc} = 16$.

5.4.3 The Confidence Thresholds of Based Deltas for Prefetch Filling into L1D and L2C. Figure 8(c) shows the speedup, L1D coverage, and L1D accuracy with different CONF_THRSHOLD_L1D for prefetching into L1D. As shown in the figure, the L1D coverage does not decrease significantly when the CONF_THRSHOD_L1D ranges from 0.5 to 0.8. In addition, as the threshold increases, the performance improvement remains relatively steady and the L1D accuracy improves, thereby reducing the cache pollution caused by inaccurate prefetching. Although CONF_THRSHOD_L1D values of 0.7 and 0.8 both provide significant performance improvements, the former provides higher L1D coverage while the latter provides higher L1D accuracy. Considering the limited bandwidth of the L1D cache, we select 0.8 as our default configuration. As shown in Figure 8(d), when the value of CONF_THRSHOD_L2C increases from 0.1 to 0.2, the performance gain of Hyperion changes from 46.5% to 50.1% due to an increase in L2C accuracy. However, when CONF_THRSHOD_L2C increases from 0.2 to 0.5, the performance gain decreases to 45.6%. This decrease is due to the fact that a higher CONF_THRSHOD_L2C results in a low L2C coverage. Therefore, we select 0.2 as our default confidence threshold for prefetch filling into L2C.

5.4.4 The Turn-off Mechanism for Dynamically Controlling Prefetch into L1D. When the real-time L1D accuracy falls below the L1D_ACCURACY_THRESHOLD, we issue prefetches for which the deltas have a confidence level exceeding CONF_THRSHOLD_L1D to fill into the larger L2 cache. As shown in Figure 8(e), the L1D accuracy increases while the L1D coverage remains high as L1D_ACCURACY_THRESHOLD varies from 0 to 0.6. This indicates that the turn-off mechanism primarily filters out inaccurate prefetch requests for L1D when L1D_ACCURACY_THRESHOLD is no greater than 0.6. However, when L1D_ACCURACY_THRESHOLD continues to increase, some useful prefetch requests into L1D are also filtered out, resulting in a decrease in L1D coverage. Therefore, we set 0.6 as our default threshold.

5.4.5 Effect of the Size of Prefetch Buffer. Hyperion utilizes a PB to buffer unissued prefetch requests when prefetch queue is full. As shown in Figure 8(f), when the size of the PB increases from 0 to 32, speedup increases from 49.2% to 50.1%. Moreover, a prefetch buffer with 64 entries offers an additional 0.04% improvement in performance over a 32-entry PB. When the size of the PB exceeds 64 entries, performance begins to decline, as some prefetch requests nearer the head of the PB are too late. Therefore, Hyperion uses 32 entries of the PB as a default configuration.

5.4.6 MAX Number of Used Deltas. Hyperion reads the confident deltas recorded in the entries of the delta table for both memory pages and PCs. Moreover, each entry in the delta table records up to 8 deltas, so Hyperion can obtain up to 16 deltas by searching the table using both the PC and the memory page. However, using too many deltas at once to generate a large number of prefetch requests may slow down on-demand access to a processor core. Therefore, we have evaluated the impact of the maximum number of utilized confident deltas on Hyperion's performance improvement. As shown in Figure 8(g), the performance gain increases from 44.2% to 49.9% when the maximum number of used deltas ranges from 2 to 8. Moreover, it remains steady when the

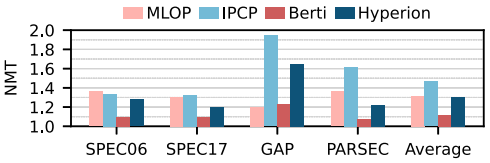

Fig. 9. Memory traffic of L1D prefetchers normalized to system without prefetcher.

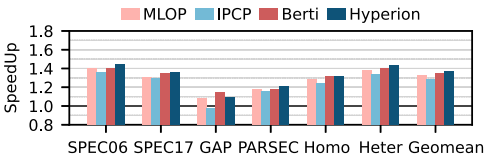

Fig. 10. Speedup of L1D prefetchers over no-prefetching in a 4-core system.

maximum number exceeds 8. We have chosen 12 as the default value for the maximum number of confident deltas, as it offers relatively higher accuracy and performance improvement. Additionally, this facilitates a direct comparison between Hyperion and Berti, as Berti also utilizes a maximum of 12 confident deltas to issue prefetch requests.

5.4.7 Effect of the Logical Page Size. Because the L1D prefetcher can see a series of virtual addresses, the logical page size used by Hyperion does not need to be consistent with the operating system page size. Moreover, the access patterns of adjacent virtual memory pages may be similar. Therefore, we explored the performance improvement of Hyperion with different sizes of the logical page. As shown in Figure 8(h), the performance improvement of Hyperion increases as the logical page size increases from 2 KB to 256 KB. The L1D accuracy increases when the logical page size increases from 4 KB to 16 KB, but then it decreases when the logical page size increases from 16 KB to 256 KB. To balance the L1D accuracy and the performance improvement, we select 32 KB as Hyperion's default logical page size.

5.5 Memory Traffic in a Single-core System

To measure memory traffic, we evaluate the **Normalized Memory Traffic (NMT)** of the evaluated prefetchers, which is defined as the ratio of the number of memory access requests to the number of requests in the baseline without prefetching. As shown in Figure 9, the average NMT of Hyperion across the four benchmark suites is 130.2%, which is 16.3% lower than that of MLOP and 1.4% lower than that of IPCP with baseline of no-prefetching. Hyperion increases memory traffic by 18.6% compared to Berti, because it issues many prefetch requests to fill into the L2C, aiming for higher coverage but resulting in lower accuracy. This leads to more useless prefetch requests alongside useful ones. In addition, Hyperion generates many useless requests, thus imposing heavy memory traffic for the irregular benchmark suite GAP due to employing more low-confidence deltas.

5.6 Performance on the Multi-core System

Figure 10 presents the multi-core performance of the prefetchers. We evaluated both homogeneous and heterogeneous mixes on a quad-core system. For homogeneous mixes, on average, Hyperion improves the performance of the non-prefetching baseline by 32.2% across all four benchmark suites. Moreover, it outperforms MLOP, IPCP, and Berti by 4.1%, 8.1%, and 0.9%, respectively. However, GAP was the only benchmark suite where Hyperion performed worse than Berti. This is because traces in GAP generally exhibit a high MPKI, and under homogeneous workload mixes, each core executes the same trace. This can lead to a situation where processor cores require a large amount of memory bandwidth at adjacent times. However, Hyperion occupies more memory bandwidth than Berti, which may delay the on-demand access requests from the processor core. Furthermore, under homogeneous workload mixes, the evaluated prefetchers exhibit lower geomean performance improvements compared to those under single-core configurations. The main reason for this phenomenon is the contention between different processor cores for LLC and

DRAM bandwidth. For evaluation on heterogeneous traces, we randomly mixed traces used in all four benchmark suites to generate 120 heterogeneous traces. On average, Hyperion improves the performance of the non-prefetching baseline by 43.7% and outperforms MLOP, IPCP, and Berti by 5.8%, 10.3%, and 3.4%, respectively. Moreover, the mixture of benchmarks from different MPKIs alleviates the contention for DRAM bandwidth, so the evaluated prefetcher achieves higher performance gains under heterogeneous workload mixes compared to homogeneous workload mixes.

6 Related Work

In addition to delta prefetchers, stride prefetchers and bit-map prefetchers also leverage the spatial locality of accesses. Stride prefetchers can learn both constant strides and complex stride sequences. Bit-map prefetchers utilize bit vectors to record access patterns of memory regions and associate these patterns with specific events. Additionally, temporal prefetchers show significant potential for applications with long-dependency memory accesses, such as online transaction processing and web applications, due to the strong temporal locality of these accesses. Furthermore, **Machine Learning (ML)** has been proven effective in prediction, and some prefetchers [6, 18, 34] have applied ML to prefetch data.

Stride prefetchers. Stride prefetchers can learn both constant strides and complex strides sequence. The IP-stride prefetcher [2] learns the strides for each load instruction. The multi-stride prefetcher [20] can detect and prefetch streams consisting of a maximum of four different strides. The more advanced prefetcher VLDP [33] uses stride sequence with varying lengths to index the next stride. SPP [24] uses signatures (hashes of strides) to recursively index the stride prediction table and thereby handle the problem of prefetch depth. Deeper prefetching may provide some timely prefetch requests. However, it also carries the potential risk of issuing requests that are either too early or too late, without an accurate latency-aware mechanism.

Bit-map prefetchers. Bit-map prefetchers utilize bit vectors to record the access patterns of memory pages and relate them to the specific events, and they issue prefetch requests based on the recorded pattern when the same event recurs. **Spatial Memory Streaming (SMS)** [37] first proposes this prediction mechanism. Compared with conventional stride prefetchers, SMS can predict complex patterns efficiently. Moreover, Bingo [4] correlates access patterns with both short and long events for higher accuracy and higher coverage, respectively. The **Dual Spatial Pattern Prefetcher (DSPatch)** [7] learns two access patterns, coverage-biased and accuracy-biased, and chooses a pattern for prefetching based on DRAM bandwidth changes. **Pattern Merging Prefetcher (PMP)** [22] finds that access patterns for memory pages that have the same trigger access page offset tend to be similar and merges these patterns to reduce storage overhead. Bit-map prefetchers can learn complex memory access patterns using bit vectors. However, they often require large storage overhead to record sufficient $<event, pattern>$ pairs, and they are not timeliness-aware compared to Hyperion.

Other prefetchers. In addition to bit-map prefetchers, temporal prefetchers, which replay the sequence of past cache misses, also require a large storage capacity to prefetch for irregular accesses [3, 10, 17, 19, 21, 23, 35, 36, 40–43]. To reduce the latency and traffic of off-chip metadata accesses, Triage [42] uses a small portion of the LLC to store important irregular accesses. The **Managed Irregular Stream Buffer (MISB)** [43] utilizes bloom filters to improve the **Irregular Stream Buffer (ISB)** [21], which caches metadata on-chip and synchronizes its contents when a TLB miss occurs. However, most of these prefetchers are only effective for irregular accesses, and the required storage is hard to provide in general processors. In recent years, a number of ML-based prefetchers have been proposed [6, 18, 34]. Pythia [6] utilizes multiple different types of program context and system-level feedback information inherent to its design based on the advantage of ML. Although Pythia has been proven to be a high-performance prefetcher, it still

has improvement room for performance gain [22]. In addition to Pythia, most prefetchers have their own strategies to control the degree of prefetching and the cache level at which the prefetch data is placed. Some general techniques, such as prefetch filters and throttling mechanisms [1, 7, 8, 12, 13, 16, 28, 29], have been proposed to improve accuracy of prefetchers.

7 Conclusion

In this article, we introduce an L1D prefetcher named Hyperion, which is designed to achieve both high coverage and accuracy. Hyperion trains deltas based on the access histories of both memory pages and PCs, enabling it to learn the diverse access patterns across numerous benchmark programs. Hyperion utilizes only high-confidence deltas to generate prefetch requests and leverages micro-architecture information along with real-time accuracy to dynamically adjust its issuing mechanism. This approach ensures its high performance and L1D accuracy. On average, Berti outperforms state-of-the-art L1D prefetchers across a wide range of applications, including SPEC CPU2006, SPEC CPU2017, GAP, and PARSEC. Moreover, the storage overhead required by Hyperion is only 4.43 KB per core.

References

[1] Jorge Albericio, Ruben Gran Tejero, Pablo Ibáñez, Víctor Viñals, and José María Llabería. 2012. ABS: A low-cost adaptive controller for prefetching in a banked shared last-level cache. *ACM Trans. Archit. Code Optim.* 8, 4 (2012), 19:1–19:20. DOI: https://doi.org/10.1145/2086696.2086698

[2] Jean-Loup Baer and Tien-Fu Chen. 1991. An effective on-chip preloading scheme to reduce data access penalty. In *Proceedings of the ACM/IEEE Conference on Supercomputing (Supercomputing'91)*, Joanne L. Martin (Ed.). ACM, 176–186. DOI: https://doi.org/10.1145/125826.125932

[3] Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2018. Domino temporal data prefetcher. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA'18)*. IEEE Computer Society, 131–142.

[4] Mohammad Bakhshalipour, Mehran Shakerinava, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Bingo spatial data prefetcher. In *Proceedings of the 25th IEEE International Symposium on High Performance Computer Architecture (HPCA'19)*. IEEE, 399–411. DOI: https://doi.org/10.1109/HPCA.2019.00053

[5] Scott Beamer, Krste Asanovic, and David A. Patterson. 2015. The GAP benchmark suite. *CoRR* abs/1508.03619 (2015).

[6] Rahul Bera, Konstantinos Kanellopoulos, Anant Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. Pythia: A customizable hardware prefetching framework using online reinforcement learning. In *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'21)*. ACM, 1121–1137. DOI: https://doi.org/10.1145/3466752.3480114

[7] Rahul Bera, Anant V. Nori, Onur Mutlu, and Sreenivas Subramoney. 2019. DSPatch: Dual spatial pattern prefetcher. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'19)*. ACM, 531–544.

[8] Eshan Bhatia, Gino Chacon, Seth H. Pugsley, Elvira Teran, Paul V. Gratz, and Daniel A. Jiménez. 2019. Perceptron-based prefetch filtering. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA'19)*, Srilatha Bobbie Manne, Hillery C. Hunter, and Erik R. Altman (Eds.). ACM, 1–13. DOI: https://doi.org/10.1145/3307650.3322207

[9] Christian Bienia, Sanjeev Kumar, Jaswinder Pal Singh, and Kai Li. 2008. The PARSEC benchmark suite: Characterization and architectural implications. In *Proceedings of the 17th International Conference on Parallel Architectures and Compilation Techniques (PACT'08)*, Andreas Moshovos, David Tarditi, and Kunle Olukotun (Eds.). ACM, 72–81. DOI: https://doi.org/10.1145/1454115.1454128

[10] Yuan Chou. 2007. Low-cost epoch-based correlation prefetching for commercial applications. In *Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'07)*. IEEE Computer Society, 301–313. DOI: https://doi.org/10.1109/MICRO.2007.39

[11] ChampSim Contributors. 2023. *ChampSim: An Open-source Trace based Simulator*. Retrieved from https://github.com/ChampSim/ChampSim

[12] Yujie Cui, Hongwei Cui, and Xu Cheng. 2023. Information leakage attacks exploiting cache replacement in commercial processors. *IEEE Trans. Comput.* 72, 9 (2023), 2536–2547.

[13] Yujie Cui, Chun Yang, and Xu Cheng. 2022. Abusing cache line dirty states to leak information in commercial processors. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA'22)*. IEEE, 82–97.

[14] DPC2. 2015. *The 2nd Data Prefetching Championship (DPC2)*. Retrieved from https://comparch-conf.gatech.edu/dpc2/
[15] DPC3. 2019. *The 3rd Data Prefetching Championship (DPC3)*. Retrieved from https://dpc3.compas.cs.stonybrook.edu/
[16] Eiman Ebrahimi, Onur Mutlu, Chang Joo Lee, and Yale N. Patt. 2009. Coordinated control of multiple prefetchers in multi-core systems. In *Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'09)*, David H. Albonesi, Margaret Martonosi, David I. August, and José F. Martínez (Eds.). ACM, 316–326. DOI: https://doi.org/10.1145/1669112.1669154
[17] Michael Ferdman and Babak Falsafi. 2007. Last-touch correlated data streaming. In *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software*. IEEE Computer Society, 105–115.
[18] Milad Hashemi, Kevin Swersky, Jamie A. Smith, Grant Ayers, Heiner Litz, Jichuan Chang, Christos Kozyrakis, and Parthasarathy Ranganathan. 2018. Learning memory access patterns. In *Proceedings of the 35th International Conference on Machine Learning (ICML'18) (Proceedings of Machine Learning Research*, Vol. 80*)*, Jennifer G. Dy and Andreas Krause (Eds.). PMLR, 1924–1933.
[19] Zhigang Hu, Margaret Martonosi, and Stefanos Kaxiras. 2003. TCP: Tag correlating prefetchers. In *Proceedings of the 9th International Symposium on High-Performance Computer Architecture (HPCA'03)*. IEEE Computer Society, 317–326.
[20] Sorin Iacobovici, Lawrence Spracklen, Sudarshan Kadambi, Yuan Chou, and Santosh G. Abraham. 2004. Effective stream-based and execution-based data prefetching. In *Proceedings of the 18th Annual International Conference on Supercomputing (ICS'04)*, Paul Feautrier, James R. Goodman, and André Seznec (Eds.). ACM, 1–11. DOI: https://doi.org/10.1145/1006209.1006211
[21] Akanksha Jain and Calvin Lin. 2013. Linearizing irregular memory accesses for improved correlated prefetching. In *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'13)*, Matthew K. Farrens and Christos Kozyrakis (Eds.). ACM, 247–259. DOI: https://doi.org/10.1145/2540708.2540730
[22] Shizhi Jiang, Qiusong Yang, and Yiwei Ci. 2022. Merging similar patterns for hardware prefetching. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO'22)*. IEEE, 1012–1026. DOI: https://doi.org/10.1109/MICRO56248.2022.00071
[23] Doug Joseph and Dirk Grunwald. 1997. Prefetching using Markov predictors. In *Proceedings of the 24th International Symposium on Computer Architecture (ISCA'97)*, Andrew R. Pleszkun and Trevor N. Mudge (Eds.). ACM, 252–263.
[24] Jinchun Kim, Seth H. Pugsley, Paul V. Gratz, A. L. Narasimha Reddy, Chris Wilkerson, and Zeshan Chishti. 2016. Path confidence based lookahead prefetching. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'16)*. IEEE Computer Society, 60:1–60:12. DOI: https://doi.org/10.1109/MICRO.2016.7783763
[25] Pierre Michaud. 2016. Best-offset hardware prefetching. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA'16)*. IEEE Computer Society, 469–480. DOI: https://doi.org/10.1109/HPCA.2016.7446087
[26] Agustín Navarro-Torres, Biswabandan Panda, Jesús Alastruey-Benedé, Pablo Ibáñez, Víctor Viñals Yúfera, and Alberto Ros. 2022. Berti: An accurate local-delta data prefetcher. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO'22)*. IEEE, 975–991. DOI: https://doi.org/10.1109/MICRO56248.2022.00072
[27] Samuel Pakalapati and Biswabandan Panda. 2020. Bouquet of instruction pointers: Instruction pointer classifier-based spatial hardware prefetching. In *Proceedings of the 47th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA'20)*. IEEE, 118–131. DOI: https://doi.org/10.1109/ISCA45697.2020.00021
[28] Biswabandan Panda. 2016. SPAC: A synergistic prefetcher aggressiveness controller for multi-core systems. *IEEE Trans. Comput.* 65, 12 (2016), 3740–3753. DOI: https://doi.org/10.1109/TC.2016.2547392
[29] Biswabandan Panda and Shankar Balachandran. 2016. Expert prefetch prediction: An expert predicting the usefulness of hardware prefetchers. *IEEE Comput. Archit. Lett.* 15, 1 (2016), 13–16. DOI: https://doi.org/10.1109/LCA.2015.2428703
[30] Seth H. Pugsley, Zeshan Chishti, Chris Wilkerson, Peng-fei Chuang, Robert L. Scott, Aamer Jaleel, Shih-Lien Lu, Kingsum Chow, and Rajeev Balasubramonian. 2014. Sandbox prefetching: Safe run-time evaluation of aggressive prefetchers. In *Proceedings of the 20th IEEE International Symposium on High Performance Computer Architecture (HPCA'14)*. IEEE Computer Society, 626–637. DOI: https://doi.org/10.1109/HPCA.2014.6835971
[31] Alberto Ros. 2019. Berti: A per-page best-request-time delta prefetcher. *The 3rd Data Prefetching Championship* (2019). https://api.semanticscholar.org/CorpusID:208008184
[32] Mehran Shakerinava, Mohammad Bakhshalipour, Pejman Lotfi-Kamran, and Hamid Sarbazi-Azad. 2019. Multi-lookahead offset prefetching. *The Third Data Prefetching Championship* (2019). https://api.semanticscholar.org/CorpusID:199570386
[33] Manjunath Shevgoor, Sahil Koladiya, Rajeev Balasubramonian, Chris Wilkerson, Seth H. Pugsley, and Zeshan Chishti. 2015. Efficiently prefetching complex address patterns. In *Proceedings of the 48th International Symposium on Microarchitecture (MICRO'15)*, Milos Prvulovic (Ed.). ACM, 141–152. DOI: https://doi.org/10.1145/2830772.2830793

[34] Zhan Shi, Akanksha Jain, Kevin Swersky, Milad Hashemi, Parthasarathy Ranganathan, and Calvin Lin. 2021. A hierarchical neural model of data prefetching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'21)*, Tim Sherwood, Emery D. Berger, and Christos Kozyrakis (Eds.). ACM, 861–873. DOI: https://doi.org/10.1145/3445814.3446752

[35] Yan Solihin, Josep Torrellas, and Jaejin Lee. 2002. Using a user-level memory thread for correlation prefetching. In *Proceedings of the 29th International Symposium on Computer Architecture (ISCA'02)*, Yale N. Patt, Dirk Grunwald, and Kevin Skadron (Eds.). IEEE Computer Society, 171–182. DOI: https://doi.org/10.1109/ISCA.2002.1003576

[36] Stephen Somogyi, Thomas F. Wenisch, Anastasia Ailamaki, and Babak Falsafi. 2009. Spatio-temporal memory streaming. In *Proceedings of the 36th International Symposium on Computer Architecture (ISCA'09)*, Stephen W. Keckler and Luiz André Barroso (Eds.). ACM, 69–80.

[37] Stephen Somogyi, Thomas F. Wenisch, Anastassia Ailamaki, Babak Falsafi, and Andreas Moshovos. 2006. Spatial memory streaming. In *Proceedings of the 33rd International Symposium on Computer Architecture (ISCA'06)*. IEEE Computer Society, 252–263.

[38] SPEC CPU. 2006. *SPEC CPU 2006 Benchmark Suite*. Retrieved from https://www.spec.org/cpu2006/

[39] SPEC CPU. 2017. *SPEC CPU 2017 Benchmark Package*. Retrieved from https://www.spec.org/cpu2017/

[40] Dennis Antony Varkey, Biswabandan Panda, and Madhu Mutyam. 2017. RCTP: Region correlated temporal prefetcher. In *Proceedings of the IEEE International Conference on Computer Design (ICCD'17)*. IEEE Computer Society, 73–80.

[41] Thomas F. Wenisch, Stephen Somogyi, Nikolaos Hardavellas, Jangwoo Kim, Anastassia Ailamaki, and Babak Falsafi. 2005. Temporal streaming of shared memory. In *Proceedings of the 32nd International Symposium on Computer Architecture (ISCA'05)*. IEEE Computer Society, 222–233. DOI: https://doi.org/10.1109/ISCA.2005.50

[42] Hao Wu, Krishnendra Nathella, Joseph Pusdesris, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Temporal prefetching without the off-chip metadata. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'19)*. ACM, 996–1008. DOI: https://doi.org/10.1145/3352460.3358300

[43] Hao Wu, Krishnendra Nathella, Dam Sunwoo, Akanksha Jain, and Calvin Lin. 2019. Efficient metadata management for irregular data prefetching. In *Proceedings of the 46th International Symposium on Computer Architecture (ISCA'19)*, Srilatha Bobbie Manne, Hillery C. Hunter, and Erik R. Altman (Eds.). ACM, 449–461. DOI: https://doi.org/10.1145/3307650.3322225

Received 5 February 2024; revised 30 April 2024; accepted 19 June 2024